

Synchronising States & Events



MCAST

Institute of Information &
Communication Technology

 david.deguara@mcast.edu.mt

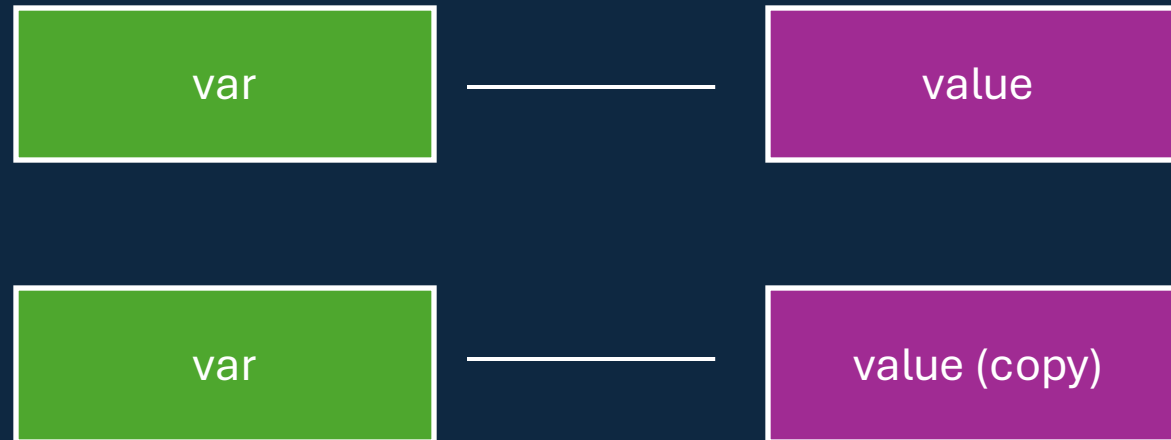


Data Types

Unmanaged (by Value) Types

Variables store their data, so they are easy to transfer over the network:

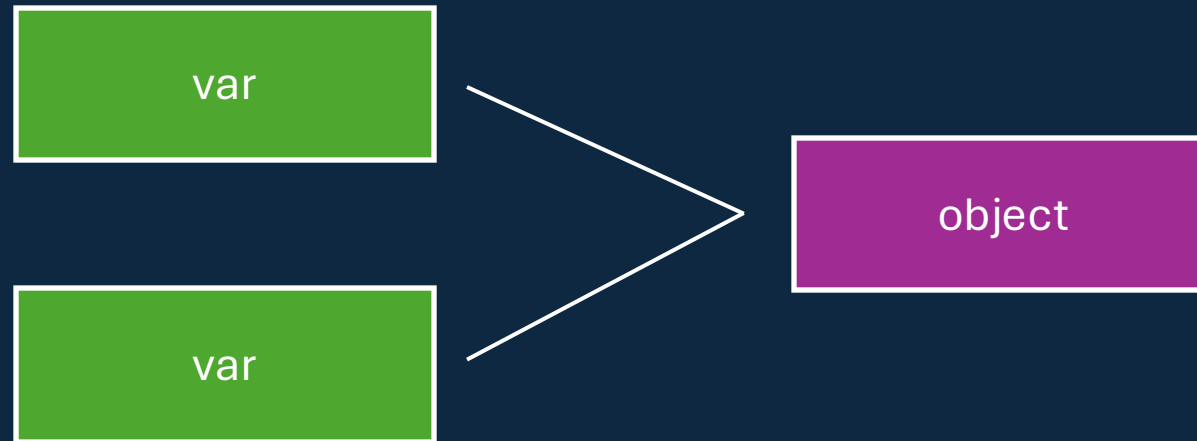
- Int
- Float
- Bool
- Enum
- Struct



Managed (by Reference) Types

Variables store references (pointers) to their data, so they cannot be transferred over the network without proper serialization.

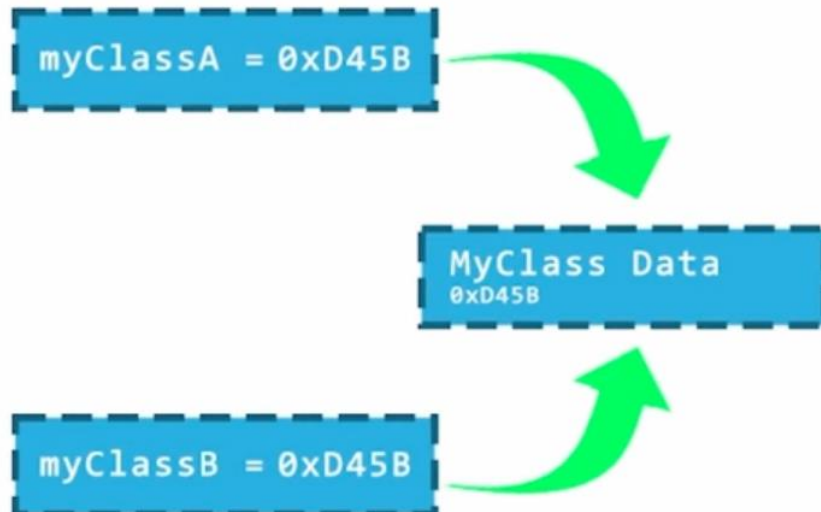
- Class
- Object
- Array
- String



Managed vs Unmanaged Types

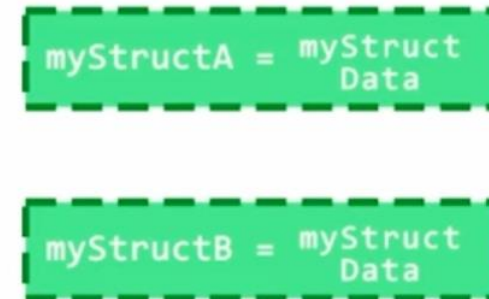
```
public class MyClass {  
    .  
}
```

```
MyClass myClassA = new MyClass();  
MyClass myClassB = myClassA;
```



```
public struct MyStruct {  
    .  
}
```

```
MyStruct myStructA = new MyStruct();  
MyStruct myStructB = myStructA;
```



Synchronising States & Events

Unity Netcode for GameObjects (Netcode) includes three options for synchronising game states and/or events:



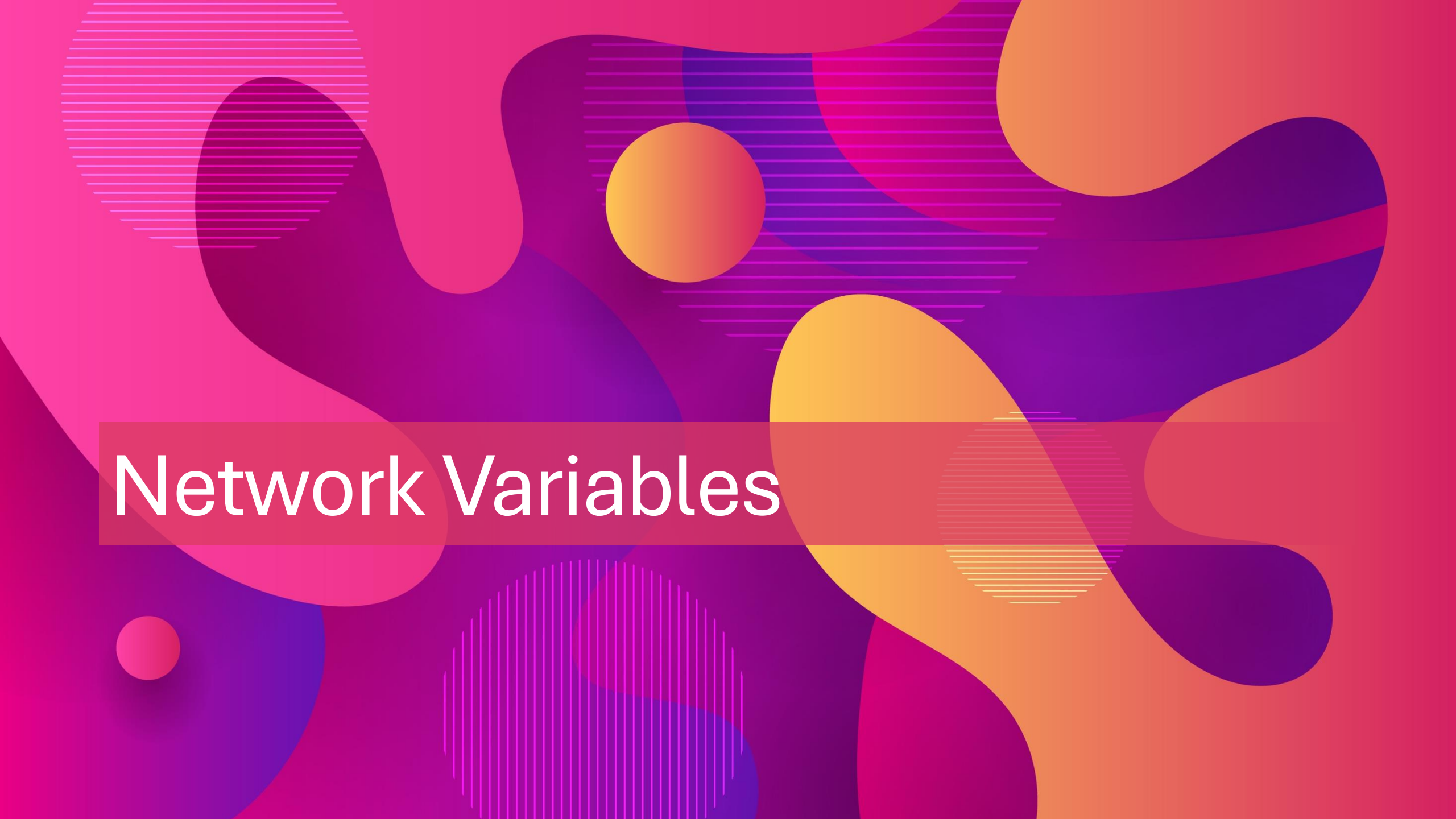
NetworkVariables

1. Handled by the "internal" messaging system and categorised under "Networking".



Messaging System

- 2. Remote Procedure Calls (RPCs)
- 3. Custom Messages

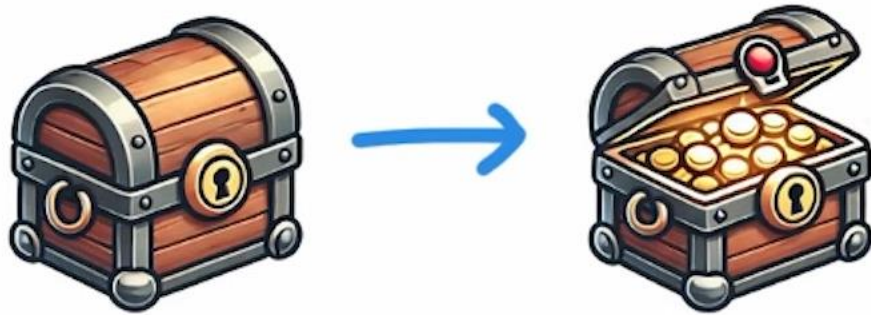


Network Variables

Network Variables

- A NetworkVariable is most commonly used to synchronise state between both connected and late joining clients.
- The NetworkVariable system only supports **non-nullable value types**, but also provides **support for INetworkSerializable** implementations as well you can **create your own NetworkVariable class** by deriving from the **NetworkVariableBase** abstract class.
- If you want something to always be synchronised with current and late-joining clients, then it's likely a good NetworkVariable candidate.

Network Variable Example



Late Join Client

Network Variable Example

```
public class PlayerNetwork: NetworkBehaviour
{
    private NetworkVariable<int> randomNumber = new
    NetworkVariable<int>(1,
    NetworkVariableReadPermission.Everyone,
    NetworkVariableWritePermission.Owner);

    public override void OnNetworkSpawn()
    {
        randomNumber.Value = Random.Range(0, 100);
    }
}
```

Network Variable Struct Example

```
public class PlayerNetwork : NetworkBehaviour
{
    public struct MyCustomData : INetworkSerializable
    {
        public int _int;
        public bool _bool;
        public string _str;

        public void NetworkSerialize<T>(BufferSerializer<T> serializer) where T : IReaderWriter
        {
            serializer.SerializeValue(ref _int);
            serializer.SerializeValue(ref _bool);
            serializer.SerializeValue(ref _str);
        }
    }
}
```

Network Variable Struct Example

```
public class PlayerNetwork : NetworkBehaviour
{
    private NetworkVariable<MyCustomData> customData = new
    NetworkVariable<MyCustomData>(
        new MyCustomData {_int = 12, _bool=true, _str = "test"},
        NetworkVariableReadPermission.Everyone,
        NetworkVariableWritePermission.Owner);
}
```

The background is a vibrant, abstract composition. It features large, overlapping organic shapes in shades of red, orange, purple, and blue. Interspersed among these shapes are patterns of fine, concentric white lines and rectangular grids of small white dots. A semi-transparent horizontal band is positioned across the middle of the image.

Remote Procedure Calls

Remote Procedure Calls



The concept of an RPC is common not only in video games but in the software industry in general.



They're ways to call methods on objects that aren't in the same executable.

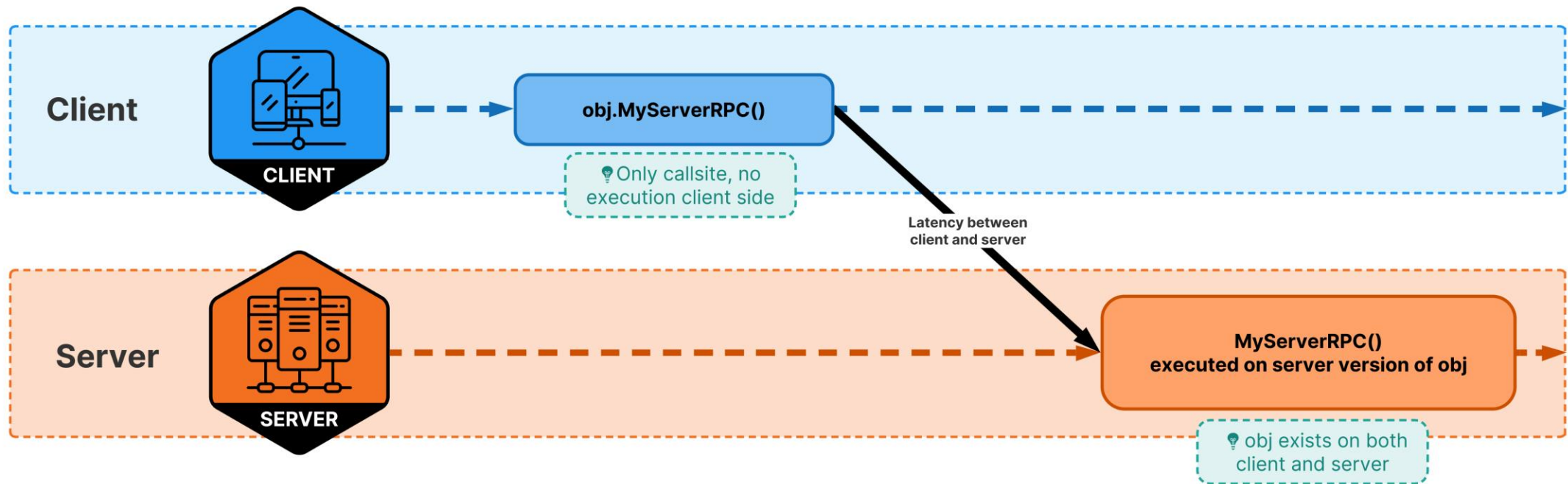
Remote Procedure Calls

- RPCs can be viewed as **a way to send an event notification** as well as **a way to handle direct communication between a server and a client** (or vice versa).
- This is sometimes useful when the ownership scope of the NetworkBehavior, that the remote procedure call is declared within, belongs to the server but you still want one or more clients to be able to communicate with the associated NetworkObject.

Server RPCs

- A **client invoked remote procedure** call received by and **executed on the server side**.
- A ServerRpc can be used by a client to notify the server that the player is trying to use a world object (that is, a door, a vehicle, etc.)

Server RPCs



Server RPC Example

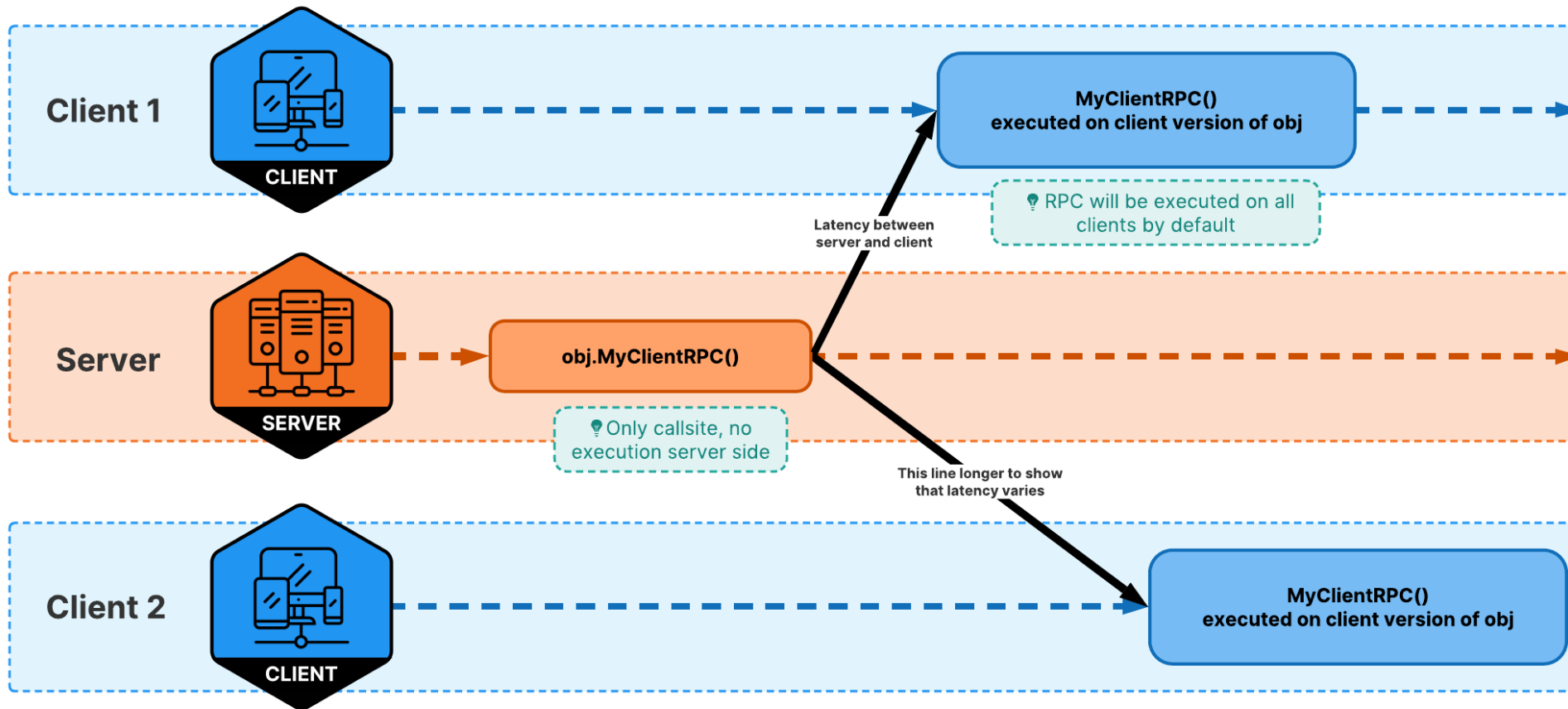
```
public class PlayerNetwork : NetworkBehaviour
{
    [ServerRpc]

    private void AdvServerRpc(ServerRpcParams serverRpcParams)
    {
        Debug.Log($"Called by {serverRpcParams.Receive.SenderClientId}");
    }
}
```

Client RPCs

- A **server invoked remote procedure call** received by and **executed on one or more clients**.
- A ClientRpc can be used by a server to notify a specific client of a special reconnection key or some other player specific information that doesn't require its state to be synchronised with all current and any future late joining client(s).

Client RPCs



Client RPC Example

```
public class PlayerNetwork : NetworkBehaviour
{
    [ClientRpc]

    private void AdvClientRpc(ClientRpcParams clientRpcParams)
    {
        Debug.Log($"Executed on all clients!");
    }
}
```

PLEASE BE MINDFUL OF OTHERS WHO ARE USING THE LAB AFTER YOU



- Plug back in power cables that are unplugged and connect HDMI / keyboard & mouse back to PCs
- Throw away any rubbish you have brought in with you outside
- Put your chair back under the desk and keep everything orderly for the person using it after you
- Report any PCs that are not working to your lecturer

THANK YOU